لإصلاحات التفصيلية لتهيئة التطبيق للإنتاج - 2-7-2025مقدمة:تهدف هذه الوثيقة إلى تقديم خطة عمل شاملة وموجهة للمبرمجين لمعالجة المشكلات المحددة في تحليل بنية التطبيق، وتحسين الأمان والأداء، وتوحيد البنية العامة، وذلك استعدادًا لإطلاق التطبيق في بيئة الإنتاج. تعتمد هذه الخطة على التوثيقات التحليلية السابقة، وستوفر المهام المحددة، والأولويات، والمراجع اللازمة لضمان تنفيذ فعال.نطاق العمل:ستركز هذه الخطة على معالجة الجوانب التالية:

1. الأمان (Security): معالجة الثغرات ونقاط الضعف الأمنية.
1. الأداء (Performance): تحسين سرعة واستجابة التطبيق.
1. البنية الهيكلية (Architectural Structure): توحيد البنية، وتحسين فصل الاهتمامات، والالتزام بأنماط التصميم (DDD, Redux, Repository Pattern).
1. إدارة متغيرات البيئة (Environment Variables Management): توحيد وتنظيم المتغيرات البيئية.
1. دمج Prisma (Prisma Integration): استكمال توحيد طبقة الوصول إلى البيانات باستخدام Prisma.

مراحل خطة العمل والأولويات:لقد تم تقسيم خطة العمل إلى ثلاث مراحل رئيسية، مع تحديد الأولويات لكل مهمة:

- المرحلة 1: الإصلاحات الحرجة والفورية (Immediate & Critical Fixes): (الأولوية: عالية جدًا)
  - الأهداف: معالجة الثغرات الأمنية الحرجة، وتحسينات الأداء الأساسية التي تؤثر على الاستقرار العام، وتصحيح التناقضات الهيكلية الرئيسية التي تعيق التطوير.
- المرحلة 2: التحسينات الهيكلية والأدائية (Structural & Performance Enhancements): (الأولوية: متوسطة إلى عالية)
  - الأهداف: تعميق فصل الاهتمامات، وتحسينات أداء إضافية، وتوحيد الأنماط البرمجية، والبدء في دمج Prisma بشكل أوسع.
- المرحلة 3: الصقل والتحسين المستمر (Refinement & Continuous Improvement): (الأولوية: متوسطة)
  - الأهداف: إكمال التحويل إلى بنية المستودع، وتنظيف التعليمات البرمجية، وتحسين التوثيق، وتطبيق ممارسات التطوير المستمرة.

المرحلة 1: الإصلاحات الحرجة والفورية

الأهداف:

- إزالة أي مخاطر أمنية مباشرة.
- تحسين الأداء للميزات الحيوية.
- توحيد إدارة متغيرات البيئة.
- حل مشاكل تكرار شرائح Redux للوكلاء.

1.1 معالجة تخزين Auth Tokens في localStorage	التفاصيل: التوقف عن تخزين رموز المصادقة (Auth Tokens) في localStorage نظرًا لتعرضها لهجمات XSS. يجب استخدام آليات أكثر أمانًا مثل HttpOnly cookies (بالتنسيق مع Next.js API routes للتعامل مع تعيين الكوكيز) أو طرق آمنة أخرى لتخزين التوكنات الحساسة. تأثير على التعديل: يتطلب تعديلات على طريقة المصادقة وتخزين التوكنات، قد يؤثر على جميع أجزاء التطبيق التي تعتمد على التوكنات من العميل.	عالية جدًا	docs\وضع التطبيق التقنى\حالة التطبيق 2-7-2025. md docs\وضع التطبيق التقنى\وضع التطبيق لكل فيتشر\إدارة وتهيئة عميل Supabase 2-7-2025.md	نظام مصادقة أكثر أمانًا لا يعتمد على localStorage لتخزين التوكنات الحساسة.
1.2 توحيد وإدارة متغيرات البيئة	التفاصيل: أ. توحيد التسميات: تغيير جميع متغيرات البيئة التي تشير إلى نفس الغرض إلى اسم موحد (مثال: NEXT_PUBLIC_SUPABASE_URL لجميع الاستخدامات العامة، و SUPABASE_SERVICE_ROLE_KEY للخادم فقط). ب. تحديث الاستخدام: تحديث جميع ملفات التعليمات البرمجية التي تستخدم هذه المتغيرات لتعكس التسميات الجديدة. ج. تنظيف ملفات .env: ضمان أن .env و .env.local يتبعان الاستراتيجية الموحدة (.env للقيم الافتراضية، .env.local للتجاوزات المحلية، وكلاهما في .gitignore). تأثير على التعديل: واسع النطاق ولكنه مباشر، يتطلب البحث والاستبدال في جميع أنحاء الكود.	عالية جدًا	docs\وضع التطبيق التقنى\ادارة_متغيرات_البيئة_2-7-2025.md	بنية موحدة لمتغيرات البيئة، وتقليل الارتباك، وتحسين الأمان.
1.3 معالجة مشاكل N+1 في الاستعلامات الحرجة	التفاصيل: تحديد وتعديل استعلامات قاعدة البيانات التي تعاني من مشكلة N+1 (خاصة في Delivery Agent Management, Order Management, Geographic Zone Management, Order and Trip Tracking). استخدم التحميل المسبق (Eager Loading) أو الاستعلامات المركبة (Joins) لتقليل عدد الاستعلامات إلى قاعدة البيانات. تأثير على التعديل: يتطلب تعديل منطق الوصول إلى البيانات في الخدمات أو المستودعات المتأثرة.	عالية جدًا	docs\وضع التطبيق التقنى\وضع التطبيق لكل فيتشر\إدارة مندوبى التوصيل 2-7-2025. md docs\وضع التطبيق التقنى\وضع التطبيق لكل فيتشر\إدارة الطلبات 2-7-2025. docs\md\وضع التطبيق التقنى\وضع التطبيق لكل فيتشر\إدارة المناطق الجغرافية 2-7-2025. docs\md\وضع التطبيق التقنى\وضع التطبيق لكل فيتشر\تتبع الطلبات والرحلات 2-7-2025.md	تحسين كبير في أداء استعلامات قاعدة البيانات، وتقليل زمن استجابة بعض الميزات.

1.4 توحيد شرائح Redux للوكلاء	التفاصيل: القضاء على الازدواجية بين `src/domains/agents/store/agentsSlice.ts` و `src/store/newAgentsSlice.ts`. دمج الوظائف في شريحة واحدة، على الأرجح في `newAgentsSlice.ts` مع نقلها إلى موقع منطقي داخل `src/store/slices/` (سيتم إنشاؤه في المرحلة 2). تأثير على التعديل: يتطلب تحديث أي مكان يستخدم الشريحة المكررة.	عالية	docs\وضع التطبيق التقنى\تحليل بنية التطبيق 2-7-2025.md	متجر Redux موحد لنطاق الوكلاء، وتقليل التعليمات البرمجية المكررة، وتحسين وضوح الحالة.
1.5 معالجة منطق كلمة المرور المؤقتة الضعيفة	التفاصيل: في `Delivery Agent Management`، تحسين آلية إنشاء كلمات المرور المؤقتة لضمان أنها قوية وعشوائية بدرجة كافية. النظر في استخدام حلول جاهزة لتوليد كلمات المرور الآمنة. تأثير على التعديل: تغييرات في وظائف إدارة المستخدمين.	عالية	docs\وضع التطبيق التقنى\وضع التطبيق لكل فيتشر\إدارة مندوبى التوصيل 2-7-2025.md	زيادة أمان حسابات المستخدمين.
1.6 إزالة التسجيلات الزائدة لرسائل الخطأ الحساسة	التفاصيل: مراجعة الكود، خاصة في `Supabase Client Management`, `Delivery Agent Management`, `Order Management`, `Waste Collection and Invoice Management`, `Order and Trip Tracking`, `Payment and Financial Management`, و `Customer Management`. إزالة أي تسجيلات (logging) لرسائل خطأ Supabase المفصلة أو بيانات حساسة قد تكشف عن معلومات أمنية. تأثير على التعديل: تغييرات طفيفة في نقاط تسجيل الأخطاء.	عالية	docs\وضع التطبيق التقنى\حالة التطبيق 2-7-2025.md docs\وضع التطبيق التقنى\وضع التطبيق لكل فيتشر\إدارة وتهيئة عميل Supabase 2-7-2025.md ... (وغيرها من وثائق الميزات)	تحسين الأمان بتقليل كشف معلومات النظام الحساسة في سجلات الأخطاء.

المرحلة 2: التحسينات الهيكلية والأدائية

الأهداف:

- **فصل طبقة النطاق عن واجهة المستخدم وتفاصيل API.**
- **تحسين تنظيم متجر Redux.**
- **تقسيم الخدمات كبيرة الحجم.**
- **تنفيذ التصفح (Pagination) للبيانات الكبيرة.**

- **اعتماد TypeScript بشكل كامل.**

المهمة	التفاصيل	الأولوية	التوثيق المرجعي	المخرجات المتوقعة
2.1 إعادة هيكلة طبقة النطاق (DDD)	التفاصيل: أ. نقل `components/` و `pages/` من داخل مجلدات النطاق (`src/domains/*/`) إلى `src/components/` و `src/app/pages/` أو `src/app/` (حسب نظام Next.js). ب. نقل مجلدات `api/` من داخل مجلدات النطاق (`src/domains/*/api/`) إلى `src/api/` (دليل جديد لـ API routes) أو دمجها في `src/services/` إذا كانت تتوافق مع منطق الخدمة. ج. التأكد من أن `src/domains/` يحتوي فقط على كيانات النطاق، كائنات القيمة، التجميعات، واجهات المستودعات، وخدمات النطاق. تأثير على التعديل: يتطلب نقل ملفات وتحديث مسارات الاستيراد في جميع أنحاء التطبيق.	عالية	`docs\وضع التطبيق التقنى\تحليل بنية التطبيق 2-7-2025.md`	بنية نطاق DDD نظيفة ومنفصلة عن طبقات العرض والبنية التحتية.
2.2 تقسيم `paymentsService.ts`	التفاصيل: تحليل وظائف `paymentsService.ts` وتقسيمها إلى ملفات خدمة أصغر وأكثر تحديدًا بناءً على مبدأ المسؤولية الواحدة (مثل `invoiceService.ts`, `transactionService.ts`, `paymentGatewayService.ts`). تأثير على التعديل: إعادة تنظيم كبيرة في `src/services/` وتحديث نقاط الاستدعاء.	متوسطة	`docs\وضع التطبيق التقنى\تحليل بنية التطبيق 2-7-2025.md` `docs\وضع التطبيق التقنى\وضع التطبيق لكل فيتشر\إدارة المدفوعات والمالية 2-7-2025.md`	خدمات أصغر وأكثر قابلية للصيانة في `./src/services`.

2.3 تنفيذ التصفح (Pagination)	التفاصيل: تطبيق آليات التصفح (Pagination) على جميع الاستعلامات التي تجلب كميات كبيرة من البيانات، خاصة في Order Management, Waste Collection and Invoice Management, Payment and Financial Management, و Customer Management. تأثير على التعديل: يتطلب تعديل وظائف جلب البيانات وتعديل واجهة المستخدم لعرض عناصر التحكم في التصفح.	متوسطة	docs\وضع التطبيق التقنى\وضع التطبيق لكل فيتشر\إدارة الطلبات docs.2025-7-2 md\ وضع التطبيق التقنى\وضع التطبيق لكل فيتشر\إدارة عمليات جمع النفايات والفواتير md .2025-7-2 docs\وضع التطبيق التقنى\وضع التطبيق لكل فيتشر\إدارة المدفوعات والمالية md .2025-7-2 docs\وضع التطبيق التقنى\وضع التطبيق لكل فيتشر\إدارة العملاء md.2025-7-2	تحسين أداء جلب البيانات الكبيرة، وتقليل استهلاك الذاكرة والنطاق الترددي.
2.4 توحيد ملفات الأدوات المساعدة واعتماد TypeScript بالكامل	التفاصيل: أ. دمج client-utils.ts و client-utils.tsx إذا كانا يخدمان نفس الغرض. ب. تحويل جميع ملفات .js المتبقية إلى .ts أو .tsx للاستفادة الكاملة من سلامة النوع. تأثير على التعديل: يتطلب بعض التحويلات وإعادة تنظيم الملفات.	متوسطة	docs\وضع التطبيق التقنى\تحليل بنية التطبيق md.2025-7-2	بيئة عمل أكثر اتساقًا وسلامة للنوع في مجلدات الأدوات المساعدة.
2.5 تحسين فصل المخاوف في Redux Store	التفاصيل: إنشاء دليل /src/store/slices وتجميع جميع شرائح Redux فيه (بما في ذلك الشريحة الموحدة للوكلاء). مراجعة كل شريحة للتأكد من أنها تلتزم بمبدأ المسؤولية الواحدة. تأثير على التعديل: يتطلب نقل ملفات وتحديث مسارات الاستيراد في src/store/index.ts.	متوسطة	docs\وضع التطبيق التقنى\تحليل بنية التطبيق md.2025-7-2	متجر Redux منظم بشكل أفضل وأسهل في التوسع والصيانة.

2.6 تحسين استعلامات select('*')	التفاصيل: مراجعة جميع الاستعلامات التي تستخدم select('*') (خاصة في Delivery Agent Management, Waste Collection, Order Tracking, Payment, Customer Management) وتحديد الحقول المطلوبة فقط. تأثير على التعديل: تغييرات في استعلامات Supabase.	متوسطة	docs\وضع التطبيق التقنى\وضع التطبيق لكل فيتشر\إدارة مندوبى التوصيل md .2025-7-2 docs\وضع التطبيق التقنى\وضع التطبيق لكل فيتشر\إدارة عمليات جمع النفايات والفواتير md docs.2025-7-2\وضع التطبيق التقنى\وضع التطبيق لكل فيتشر\تتبع الطلبات والرحلات md docs.2025-7-2\وضع التطبيق التقنى\وضع التطبيق لكل فيتشر\إدارة المدفوعات والمالية md docs.2025-7-2\وضع التطبيق التقنى\وضع التطبيق لكل فيتشر\إدارة العملاء md.2025-7-2	تقليل حجم البيانات المسترجعة، وتحسين الأداء، وتقليل استهلاك الموارد.

المرحلة 3: الصقل والتحسين المستمر

الأهداف:

- **تطبيق نمط المستودع بشكل كامل عبر التطبيق.**
- **معالجة التبعيات الدائرية والتعليمات البرمجية المكررة.**
- **توحيد آليات معالجة الأخطاء والتحقق من صحة المدخلات.**
- **تحسين التوثيق المعماري وتوحيد معايير التعليمات البرمجية.**

المهمة	التفاصيل	الأولوية	التوثيق المرجعي	المخرجات المتوقعة
3.1 تنفيذ طبقة المستودع (Repository Layer)	التفاصيل: إنشاء دليل ./src/repositories. تحديد واجهات المستودعات في طبقة النطاق (/src/domains/*/) وتنفيذها باستخدام Prisma Client في طبقة البنية التحتية (/src/repositories/). نقل جميع تفاعلات قاعدة البيانات تدريجيًا من Supabase client API المباشر إلى Prisma عبر المستودعات. تأثير على	متوسطة	docs\وضع التطبيق التقنى\تحليل بنية التطبيق md .2025-7-2 docs\وضع التطبيق التقنى\ادارة_Prisma_2-7-2025.md	فصل كامل بين منطق النطاق وتفاصيل قاعدة البيانات، وتحسين قابلية الاختبار والصيانة.

	التعديل: إعادة هيكلة جوهرية لطبقة الوصول إلى البيانات.			
3.2 معالجة التبعيات الدائرية والتعليمات البرمجية المكررة	التفاصيل: إجراء تحليل للتبعيات لتحديد أي تبعيات دائرية وحلها باستخدام مبادئ التصميم المناسبة (مثل Dependency Inversion). تحديد التعليمات البرمجية المكررة ونقلها إلى وظائف أو مكونات مساعدة قابلة لإعادة الاستخدام في `src/utils/` أو `src/shared/`. تأثير على التعديل: تغييرات على مستوى الكود لتقليل الاقتران والتكرار.	متوسطة	docs\وضع التطبيق التقنى\تحليل بنية التطبيق 2-7-2025.md	بنية كود أكثر نظافة، وتقليل التعقيد، وتحسين قابلية الصيانة.
3.3 توحيد آليات معالجة الأخطاء والتحقق من صحة المدخلات	التفاصيل: إنشاء نظام مركزي لمعالجة الأخطاء على مستوى التطبيق (خاصةً الأخطاء التي تظهر للمستخدم). تطبيق التحقق من صحة المدخلات الصارم (Input Validation) في طبقة التطبيق أو النطاق لجميع البيانات الواردة. تأثير على التعديل: يتطلب تعديل نقاط إدخال البيانات ومعالجة الأخطاء الحالية.	متوسطة	docs\وضع التطبيق التقنى\وضع التطبيق لكل فيتشر\إدارة وتهيئة عميل Supabase 2-7-2025.md docs\وضع التطبيق التقنى\وضع التطبيق لكل فيتشر\إدارة مندوبى التوصيل 2-7-2025.md docs\وضع التطبيق التقنى\وضع التطبيق لكل فيتشر\إدارة العملاء 2-7-2025.md docs\وضع التطبيق التقنى\وضع التطبيق لكل فيتشر\إدارة المدفوعات والمالية 2-7-2025.md	تحسين تجربة المستخدم من خلال رسائل خطأ واضحة، وزيادة أمان التطبيق من خلال التحقق من صحة المدخلات.
3.4 توحيد معايير التعليمات البرمجية وأتمتة نظام الوبر (Linters)	التفاصيل: مراجعة وتحديث ملفات تكوين ESLint و Prettier لفرض معايير أسلوب التعليمات البرمجية المتسقة. دمج أدوات الوبر في خطوط أنابيب CI/CD (إذا كانت متوفرة) لضمان تطبيق المعايير تلقائيًا. تأثير على التعديل: قد يتطلب إصلاحات أولية للتعليمات البرمجية لتتوافق مع المعايير الجديدة.	متوسطة	docs\وضع التطبيق التقنى\تحليل بنية التطبيق 2-7-2025.md	تعليمات برمجية متسقة ونظيفة، وتقليل الأخطاء البشرية، وتحسين جودة التعليمات البرمجية.

3.5 مراجعة وتوحيد وثائق إعادة الهيكلة	التفاصيل: دمج جميع ملفات Markdown المتعددة المتعلقة بخطط إعادة الهيكلة السابقة في مستند معماري واحد "حي" ومحدث باستمرار (مثل docs\وضع التطبيق التقنى\الوثيقة_المعمارية_الرئيسية.m d>). تأثير على التعديل: عمل توثيقي.	متوسطة	docs\وضع التطبيق التقنى\تحليل بنية التطبيق 2-7-2025.md docs\وضع التطبيق التقنى\ادارة_متغيرات_البيئة_2-7-2025.md docs.وضع التطبيق التقنى\ادارة_Prisma_2-7-2025.m d> (وجميع وثائق إعادة الهيكلة الأخرى في الجذر و src/)	مصدر واحد للحقيقة لجميع القرارات والخطط المعمارية، وتحسين فهم الفريق للبنية.

ملاحظات هامة للمبرمجين:

- **التواصل المستمر: يجب على المبرمجين التواصل بشكل مستمر مع مدير المشروع وDevOps خلال تنفيذ هذه الخطة للإبلاغ عن أي تحديات أو عقبات.**
- **اختبار الوحدة والتكامل: لكل مهمة يتم تنفيذها، يجب كتابة أو تحديث اختبارات الوحدة (Unit Tests) واختبارات التكامل (Integration Tests) لضمان أن التغييرات لا تُدخل انحدارات (regressions).**
- **مراجعات الكود (Code Reviews): يجب أن تخضع جميع التغييرات لمراجعات كود صارمة لضمان الجودة والالتزام بالمعايير المعمارية والأمنية الجديدة.**
- **المرونة: هذه الخطة هي دليل إرشادي. قد تظهر تحديات غير متوقعة تتطلب تعديلات. كن مستعدًا للتكيف مع التغييرات بالتشاور مع مدير المشروع.**
- **الأمان أولاً: يجب دائمًا إعطاء الأولوية القصوى للأمان في جميع مراحل التطوير والتعديل.**
-